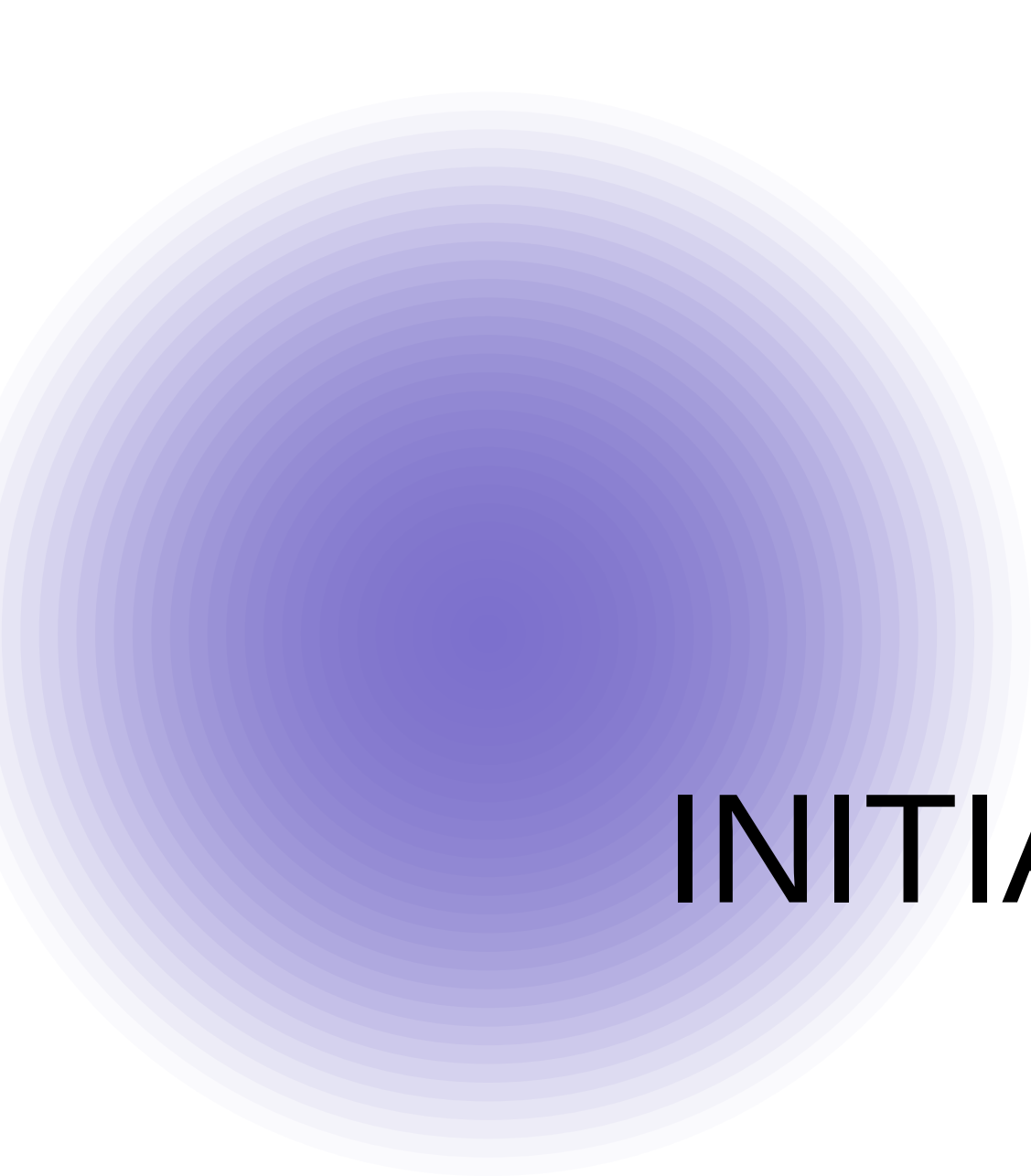




INITIAL ACCESS AGENT

# TWO-PASS DESIGN

## Pass 1 — Zeek Fast Scan (no PCAP)

- Parses zeek.rdp log for brute-force count & session bytes
- Populates InitialAccessFindings used by needs\_pcap\_drilldown()
- Runs for all 9 days — cheap and fast

## Pass 2 — LLM ReAct Loop (tshark)

- Triggered only on days flagged by Pass 1
- ForensicAgent drives up to MAX\_AGENT\_STEPS = 30 tshark calls
- Uses Azure OpenAI function-calling (not LangGraph tools)
- Produces full Markdown forensic report

- Rationale: Pass 1 filters 9 days to 1–2; Pass 2 only runs
- where evidence exists — minimises LLM cost & runtime

`initial_access_agent.py`

```
# initial_access_agent.py
MAX_AGENT_STEPS = 30
MAX_OUTPUT_CHARS = 15_000

# Pass 1: deterministic Zeek scan
```

# SYSTEM PROMPT — REASONING PRINCIPLES

- Brute-force source IPs are NOT the successful attacker
  - The actual login may come from a different IP using stolen creds
- Session size is the key discriminator
  - Failed RDP = 2–5 KB, ~20 frames, < 10 seconds
  - Successful RDP = 50–500+ KB, hundreds of frames, minutes
- Sort TCP conversations by bytes — the successful
- session is a visible outlier among hundreds of failures
- Check for pre-existing compromise
  - C2 beacons / RMM tool DNS queries active from capture start
  - Means network was already breached before the capture window

SYSTEM\_PROMPT (initial\_access\_agent.py)

```
SYSTEM_PROMPT = """  
You are an expert network forensic analyst  
specialising in initial access vector identification.
```

CRITICAL ANALYSIS PRINCIPLES

# SEED QUERIES

- Auto-executed before the ReAct loop starts
- Prevents the LLM from missing critical upfront evidence

## 1. PCAP Overview

- Capture metadata: time range, packet count, file size

## 2. RDP Brute-Force Sources

- group\_count: SYNs to port 3389, grouped by source IP
- Identifies all brute-force participants

## 3. TCP Conversations on Port 3389

- tcp\_conversations sorted by bytes (top 50)
- The successful session is the visible outlier

## 4. C2 / RMM Tool DNS Queries

- packet\_query: dns.qry.name contains rmm/tactical/mesh
- Detects pre-existing C2 beacons from capture start

### SEED\_QUERIES (initial\_access\_agent.py)

```
SEED_QUERIES = [  
    {  
        'label': 'PCAP Overview',  
        'tool': 'pcap_overview',  
        'args': {},
```

# OUTPUT SCHEMA — InitialAccessFindings

- Structured dataclass written to PipelineState
- Consumed by Lateral Movement, Payload agents & report writer

## Key fields:

- patient\_zero — first compromised host IP
  - attack\_vector — e.g. 'RDP brute-force'
  - attacker\_ip — IP that made the successful login
  - exposed\_service — e.g. 'RDP/3389'
  - brute\_force\_count — number of failed attempts
  - successful\_session\_bytes — size of the winning session
  - session\_start / session\_end — ISO-8601 timestamps
  - pre\_existing\_compromise — bool (C2 already active?)
  - iocs — List[IOC] with confidence levels
- Fields also feed needs\_pcap\_drilldown() thresholds:
    - successful\_session\_bytes > 10,000 → flag for PCAP drill-down
    - brute\_force\_count > 1,000 → flag for PCAP drill-down

shared/data\_contract.py

```
@dataclass
class InitialAccessFindings:
    """Output schema for the Initial Access agent."""
    summary: str = ''
    patient_zero: Optional[str] = None
```

# ADAPTER — MARKDOWN TO STRUCTURED DATA

- ForensicAgent outputs free-form Markdown — not JSON
- initial\_access\_adapter.py bridges this to PipelineState

## **\_parse\_report()** extracts via regex:

- Patient Zero — looks for 'Patient Zero: <IP>'
  - Attacker IP — 'successful login from <IP>'
  - Exposed port — 'port <number>'
  - Brute-force count — '<N> SYN / failed'
  - Session bytes — first '<N> bytes' mention
  - Pre-existing C2 — keyword: 'pre-existing', 'c2 beacon'
  - IOCs — all IPs in the IOC section of the report
- Why regex not structured output?
    - ForensicAgent uses raw Azure OpenAI SDK (not LangChain)
    - submit\_report tool accepts free Markdown — simpler prompt
    - Regex post-processing is more robust than forcing JSON mid-loop

agents/initial\_access\_adapter.py

```
# agents/initial_access_adapter.py
```

```
def _parse_report(report: str) -> InitialAccessFindings:  
    findings = InitialAccessFindings(  
        report_markdown=report,
```